

CS383 - Software Engineering

Software Design (structural modeling)

Outline



- Structural modeling
 - Class diagrams
 - Object diagrams

Structural modeling



- Describes the system underdevelopment components (packages), entities, properties (attributes and operations) and relationships.
- Structural models can be static models or dynamic
 - Static: describes the system structure (entities and relationship)
 - Dynamic: describes the components of the system at execution
- Structural models are helpful when designing the architecture of the system

UML Structural models



- UML offers a number of structural models
 1. Class diagram (requested in your project)
 2. Object diagram
 3. ... and more (see M. Fowler “UML Distilled”)

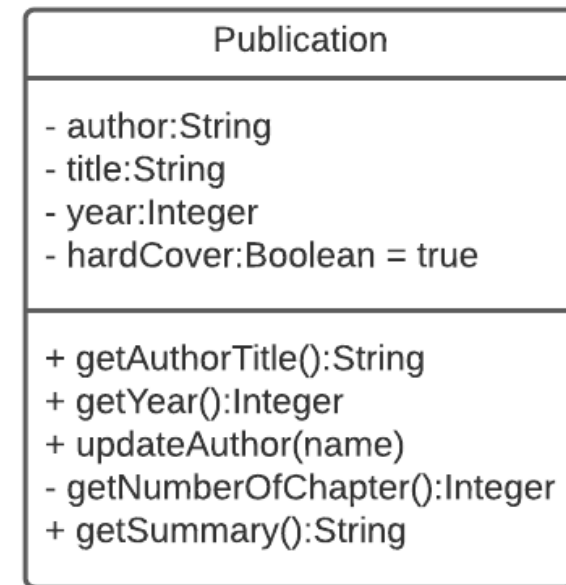
Class diagram



- Describes a set of entities (as classes), properties and relationship between properties
- Classes are fundamental to object-oriented languages and programs
- Class models/diagrams are fundamental to object-oriented designs
 - they capture the abstract structure of the classes in the system
 - they capture the relationships/associations between the classes

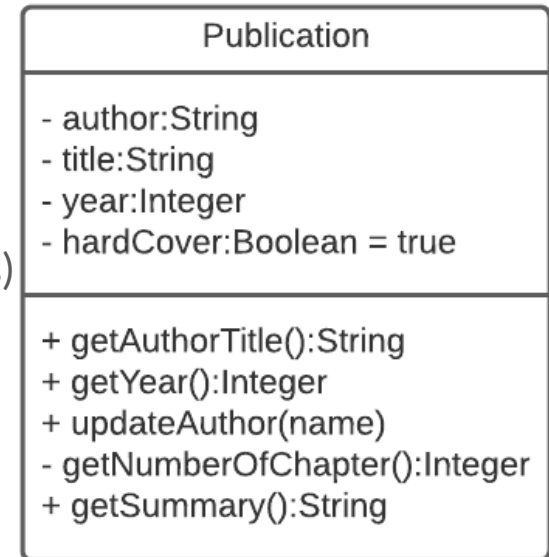
Class diagram (cont.)

- Entity/class describes a group of object that have properties.
- Properties/members can be
 - Attributes: type data held by an instance of the class
 - Types: Integer, Real (for floating point numbers), Boolean, String
 - Operation: function applied to/by an instance of the classes. Operation have parameters and return type
 - Relationship to other classes/entities



Class diagram (cont.)

- Properties can be annotated
 - Types and return types
 - Visibility: + public, - private, #protected (accessible within the class and any subclass i.e. inherited classes)
 - Stereotype that indicate the kind of class (interface, abstract)
 - Use <<interface>>, <<abstract>> before/above the name
 - A class can **generalize** a class or abstract class. But, it can **realize** an interface



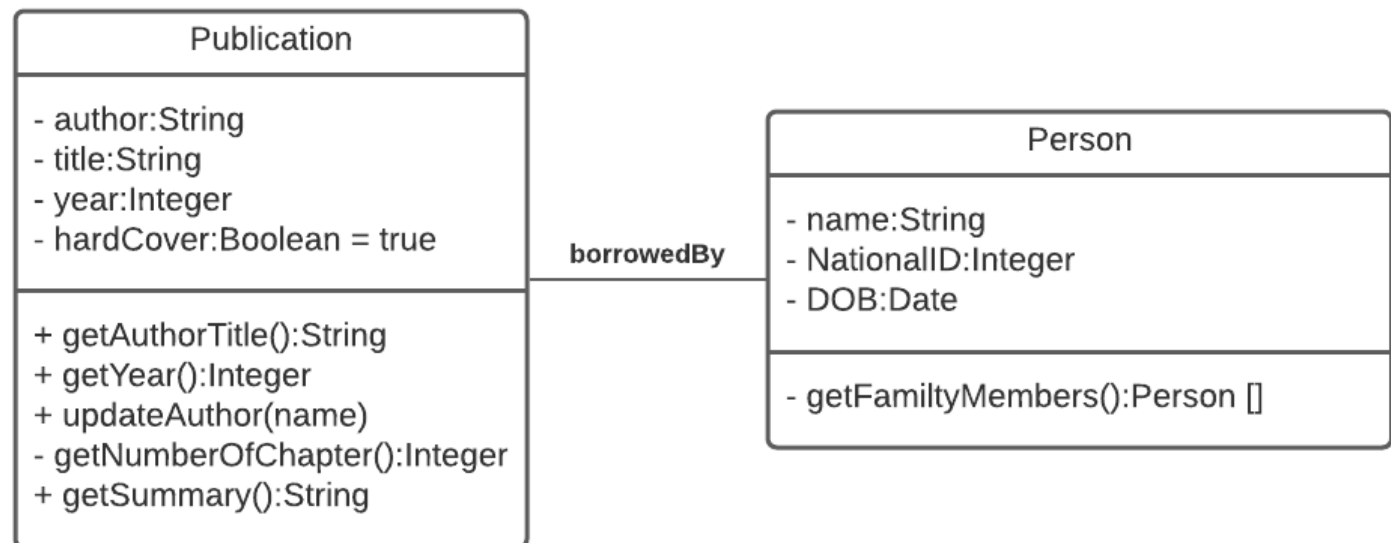
Class diagram (cont.)



- Non-instantiatable classes can be
 - Abstract: to be inherited by instantiatable classes. It can have implemented operations
 - Interface: provides only prototypes of operations. No implementations.
 - Cannot be generalized (inherited), but can be realized (implemented).
- If a class has a superclass or implements an interface, then
 - graphical representation must illustrate that, or
 - textually representation is used inside of the class

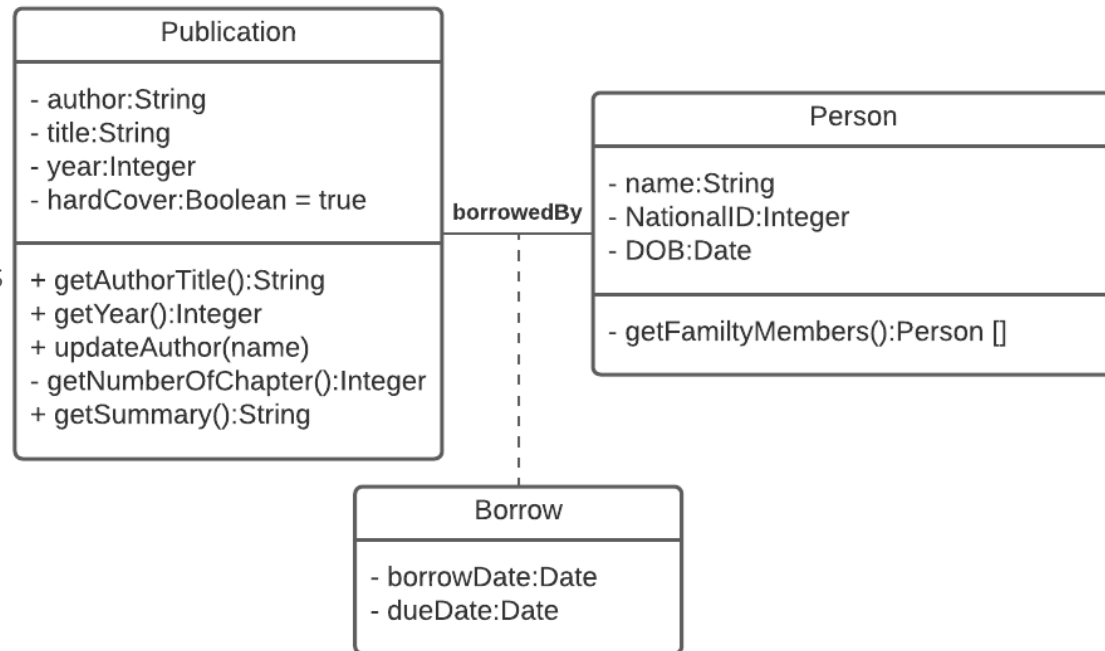
UML association

- Indicates usually a bidirectional relationship between classes.
- Association can have rules
 - Near publication: item
 - Near Person: borrower
- Association can have multiplicity
 - Near publication: 0..* and near Person: 1



UML association (cont.)

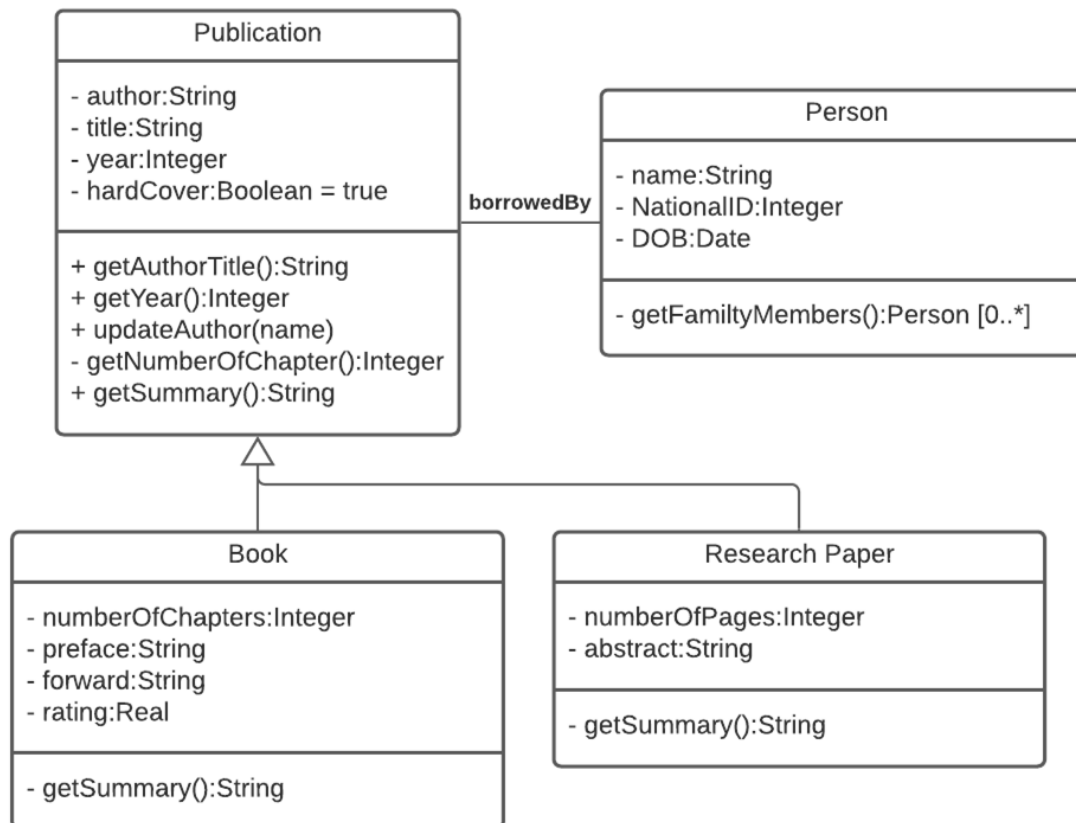
- Association can have even properties
- UML displays these properties as association classes



- An association class can be transformed into independent class with two relations
- In the example, we can a class Borrow and add the following
 - attribute: `numberOfRepeat:Integer`
 - Operations: `repeatBorrow():Boolean`

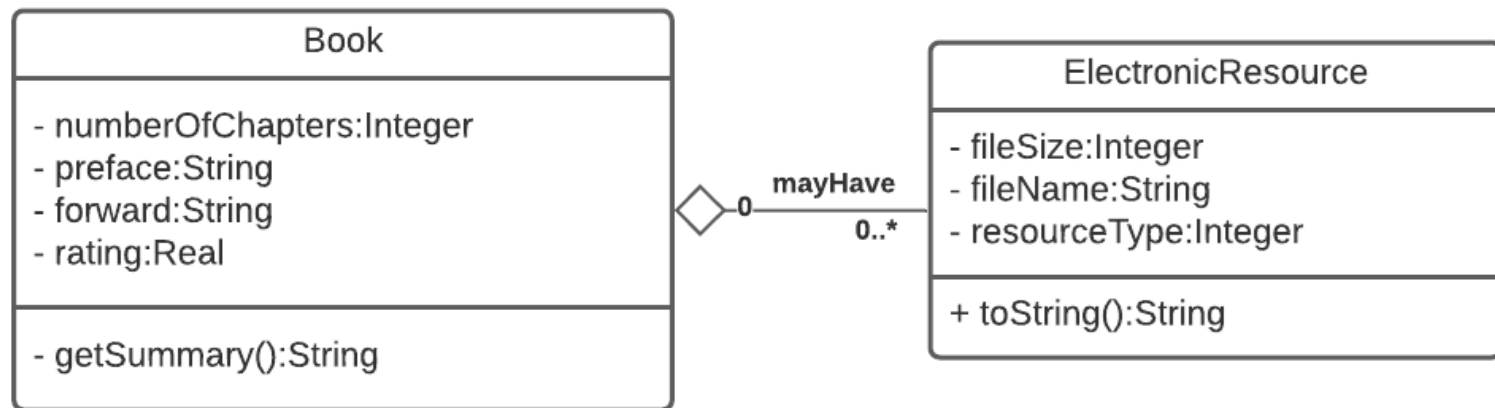
UML generalization

- Generalization is basically inheritance relation
 - Parents are referred to as superclass
 - Children are referred to as subclasses or specifications
 - UML support multiple inheritance (a class inherits from two different parents)



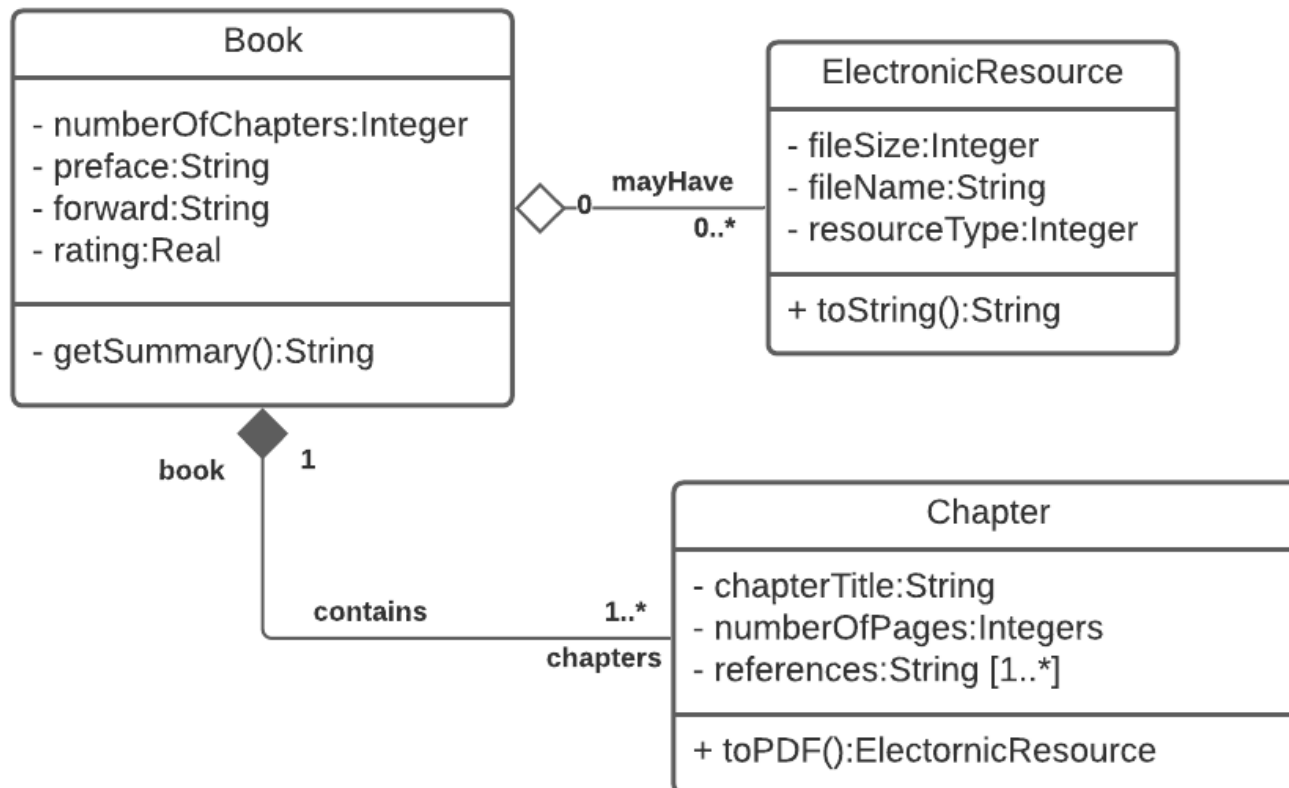
UML association extensions

- Aggregation: an association that indicates that one class is a component of another. But not strongly dependent.



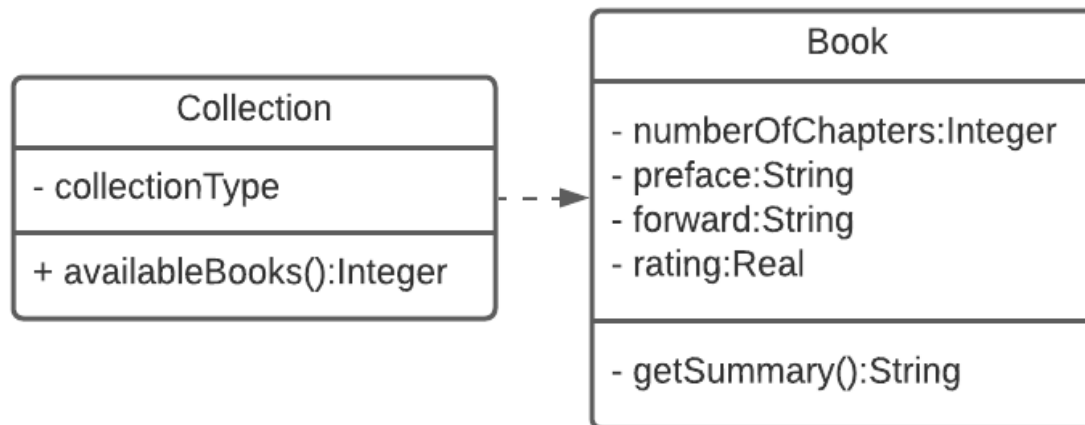
UML association extensions (cont.)

- Composition: works like aggregation but the two classes are strongly



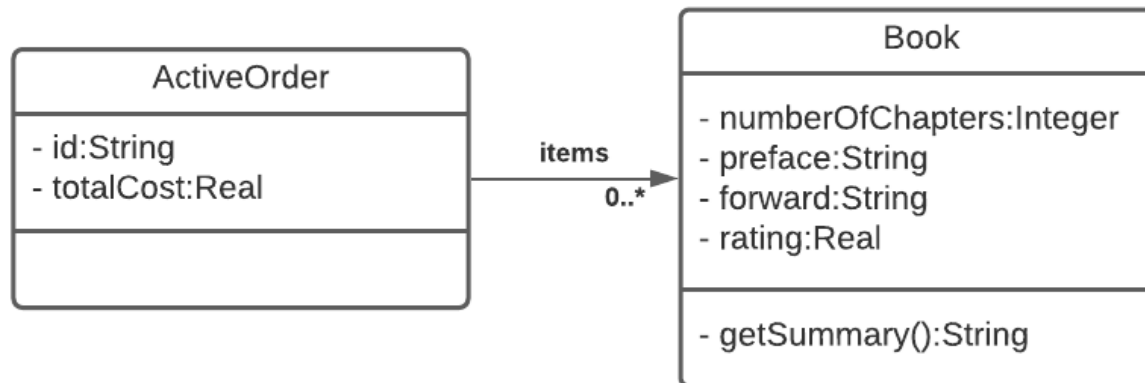
UML association extensions (cont.)

- Dependency: shows that an object content depends on another object



UML association extensions (cont.)

- Navigation association: describes that relationship of two classes is only traversed in one direction



Class diagram exercise 1

- Create a class diagram that describes the structure of a store (with name and address) that contains INF but at least one item (with details name, information, and price). The store also contains INF order with basic details like list of items and their quantities.
- Items can be categorized into three types: appliance, device, and cloth
- All attributes must all be specified as private attributes
- Operations
 - Store: getName() that returns type string, updateAddress(), that accepts a new address as string
 - Item: getInfo() that returns type string, getPrice() that return real type
 - Order: addItem() that accepts a new Item and returns a List type of item, getTotal() that returns type real.

Class diagram exercise 2



A hospital CEO wants to control hospital resources and how the staff of the hospital access them. He/she hires you and wants to construct a model structure that describes your proposed solution. There are many resources and they can be group into two types (although they can have common properties):

- Hardware: computers, medical equipments and coffee machines
- Software: computer applications

The staff can be medical staff or civil staff. Medical staff has access to everything including medical equipments, which is done using medical permits that can be renewed annually. Civil employee can access everything except medical equipments.

A computer hardware can consists of a number of INF software

Class diagram exercise 3



A company needs to register all its employees' personal transportations that they are using to come work. The company needs to track parking permits provided to vehicles. There are two types of vehicles allowed, compact and economy cars. Each registered car has a permit linked to it and there is always one parking spot for each permit. Permits are encoded into a string and each parking spot has a number.

Any permit can be update by calling the function `updatePermit(p)`.

Class diagram exercise 4



You are to model a representation of a tree data structure of objects. There are two types of trees: binary and nonBinary, but they share a common attribute 'label' of the object type, with 'null' as the default value.

A binary tree may have two objects of binary trees: firstChild and secondChild. The multiplicity of these two associations must be optional.

A nonBinary tree has zero or more objects of nonBinary trees, and this is represented by an association named 'children'.

An object class can be either a StringObject class with a string attribute named 'value' or a RealObject class with a real attribute named 'value'.

Provide a class diagram that illustrates the above structure. Identify the appropriate relationships between the classes. All attributes of classes must be made public.

Object diagram

- Object diagram are instances of class diagrams
- Instances can have no name, but must have type
- Names are underlined

